

Práctica 6: Clasificación de Barcos y Transferencia de Aprendizaje

10 de julio de 2023

Dataset: Juego de Aprendizaje Profundo: Dataset de Barcos
<https://www.kaggle.com/datasets/arpitjain007/game-of-deep-learning-ship-datasets>

1. Objetivo de la práctica.

Utilizar un modelo preentrenado para realizar clasificación de barcos, dadas las características que se muestran de ellos en un conjunto de imágenes.

2. Conceptos.

Seguiremos usando algunos términos que ya se han visto en prácticas anteriores, así que en seguida se muestra un nuevo concepto, el cv2.

cv2 es un módulo de Python para usar OpenCV (Open Source Computer Vision Library) es una biblioteca que contiene más de 2,500 algoritmos optimizados para procesamiento de imágenes y visión por computadora.

Como parte de sus funciones, cv2 nos permite manipular imágenes; leerlas, escribirlas, transformarlas (rotaciones, redimensiones, aplicación de filtros)

3. Herramientas a usar.

- Computadora con acceso a internet.
- Los siguientes programas y librerías:
 - Python versión 3.8.8
 - TensorFlow versión 2.6.0
 - Anaconda versión 4.10.1
 - Jupyter-lab 3.014
 - NumPy versión 1.21.4

- Pandas versión 1.2.4
- Matplotlib versión 3.3.4
- Zipfile versión 3.2.0
- Seaborn 0.11.2
- cv2 4.6.0

4. Desarrollo.

4.1. Entender el problema.

En esta ocasión, lo que se busca es hacer una clasificación multiclases en la cual dada una imagen (en este caso de barcos) se pueda identificar qué tipo de barco es. Si bien, este planteamiento es similar a lo que se ha visto en prácticas anteriores, ahora se tiene un nuevo grado de dificultad ya que se tiene como objetivo identificar barcos dadas características más específicas.

4.2. Criterio de evaluación.

Se usará la exactitud (accuracy), precisión (precision) y matriz de confusión como criterios de evaluación.

4.3. Preparar los datos.

El primer paso es importar todas las librerías con las que estaremos trabajando durante la práctica.

```
1 import numpy as np
2 import pandas as pd
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5 import os
6 import cv2
```

Se tienen 8932 imágenes en el conjunto de datos. 6252 de las imágenes están separadas para el entrenamiento y solo esas imágenes están etiquetadas. Se utilizarán esas 6252 imágenes etiquetadas y se dividirán como datos de entrenamiento, validación y prueba.

En la primera parte, se importa el archivo csv que contiene los nombres de las imágenes y las categorías de barcos correspondientes 1.

```
1 Path = "../Practica_5/" #Depende de la ruta en tu equipo
2
3 data_csv = pd.read_csv(Path + 'train/train.csv')
4 print('Data shape: ', data_csv.shape)
5
6 categories = {0: 'Cargo', 1: 'Military', 2: 'Carrier', 3: 'Cruise', 4:
7               'Tankers'}
```

```

8 # I want classes to start from 0. So I subtracted 1 from the
   categories
9 data_csv['category'] = data_csv['category'] - 1
10 data_csv['label'] = data_csv['category'].map(categories)
11 data_csv['label'] = pd.Categorical(data_csv['label'])
12 data_csv.head(5)

```

	image	category	label
0	2823080.jpg	0	Cargo
1	2870024.jpg	0	Cargo
2	2662125.jpg	1	Military
3	2900420.jpg	2	Carrier
4	2804883.jpg	1	Military

Figura 1: Ejemplo de las categorías de los barcos, según el contenido del dataset

Como buena práctica, se revisa la distribución de las imágenes a través de las 5 categorías que se están presentando en el paso anterior

```

1 sns.countplot(data_csv['label'])
2 plt.title('Ship Category Distribution')
3 plt.xlabel('Categories')
4 plt.ylabel('Counts of Observations');

```

Al ejecutarlo, se tiene esta gráfica 2

Como se puede ver en el gráfico, el conjunto de datos no se distribuye equitativamente entre las categorías. No se modificarán las cantidades de imágenes en el conjunto de datos, es preferible usar todas las imágenes para el entrenamiento, de esta forma se minimiza el riesgo a caer en un error de sobreentrenamiento

Aclarado lo anterior, se procede a usar cv2. Lo primero es definir la ruta a los datos para poder acceder a ellos

```

1 path = r'C:\Users\brugi\Documents\Diplomado\ejercicios\Practica_5\
   train\images'

```

Luego, se toma el nombre de cada imagen contenida en el archivo csv y se extrae en un bucle para que la computadora cargue la imagen correspondiente.

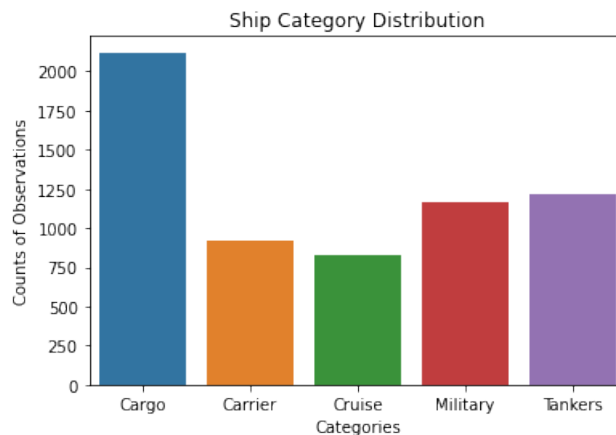


Figura 2: Distribución de las diferentes categorías que se tienen en el dataset

Posteriormente, cada imagen se convierte a formato RGB y se cambia su tamaño para que tenga las mismas dimensiones. Por último, cada imagen se guarda en una lista en formato de matriz Numpy.

```

1 img_list = list(data_csv['image'])
2
3 data_img = []
4
5 for each in img_list:
6     # Each image path
7     each_path = os.path.join(path, each)
8     # Read each image
9     each_img = cv2.imread(each_path, cv2.IMREAD_UNCHANGED)
10    # OpenCv default color is BGR. Convert it to RGB
11    each_img = cv2.cvtColor(each_img, cv2.COLOR_BGR2RGB)
12    # Resize the images
13    each_img_resized = cv2.resize(each_img, (75,75))
14    # Save arrays to a list
15    data_img.append(each_img_resized)
16
17 # Converting list to numpy array
18 X = np.array(data_img)
19 print('Shape of X: ', X.shape)

```

Al final se extraen las etiquetas del archivo csv y se aplica One Hot Encoder para tener una característica de destino.

El OneHotEncoder transforma variables categóricas en un formato que puede ser interpretado por modelos de aprendizaje automático. Específicamente, convierte cada categoría en un vector binario único (one-hot encoding), donde cada vector tiene un valor de 1 en la posición correspondiente a la categoría presente y 0 en todas las demás posiciones. [Google Docs Referencia](#)

```

1 from sklearn.preprocessing import OneHotEncoder

```

```

2
3 y = OneHotEncoder(dtype='int8', sparse=False).fit_transform(
4     data_csv['category'].values.reshape(-1,1))
5 print('Shape of y: ', y.shape)

```

Ahora se muestran imágenes aleatorias del conjunto de datos para darse una idea de cómo se categorizan los barcos 3

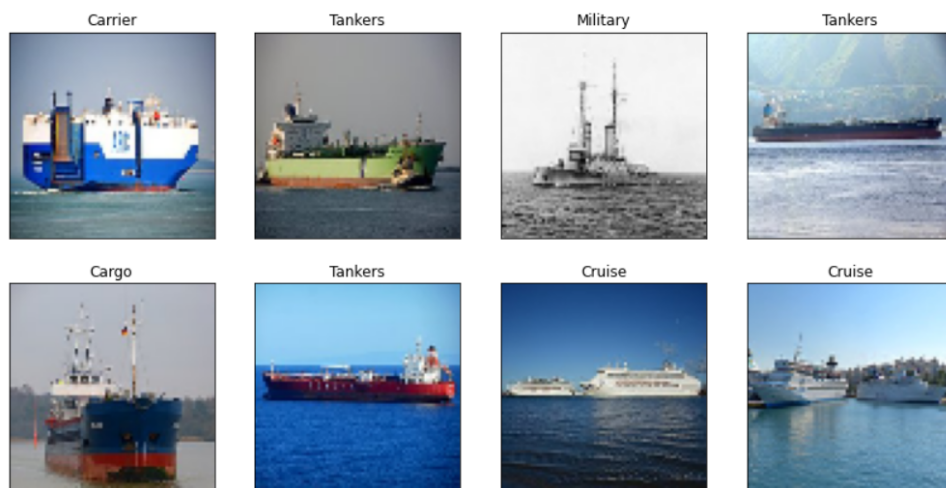


Figura 3: Imágenes ejemplo del conjunto de prueba.

4.4. División de entrenamiento, validación y prueba.

El conjunto de datos se divide en divisiones de entrenamiento, validación y prueba.

```
1 from sklearn.model_selection import train_test_split
2
3 X_data, X_test, y_data, y_test = train_test_split(X, y, test_size
4         =0.15, random_state=42)
5 X_train, X_val, y_train, y_val = train_test_split(X_data, y_data,
6         test_size=0.2, random_state=42)
7
8 print('X_train shape: ', X_train.shape)
9 print('y_train shape: ', y_train.shape)
10 print('X_val shape : ', X_val.shape)
11 print('y_val shape : ', y_val.shape)
12 print('X_test shape : ', X_test.shape)
13 print('y_test shape : ', y_test.shape)
```

```
X_train shape: (4251, 75, 75, 3)
y_train shape: (4251, 5)
X_val shape : (1063, 75, 75, 3)
y_val shape : (1063, 5)
X_test shape : (938, 75, 75, 3)
y_test shape : (938, 5)
```

Figura 4: Se puede apreciar la cantidad de datos que componen los conjuntos de datos de entrenamiento, validación y prueba.

4.5. Aumento de datos (Data Augmentation)

El aumento de datos se aplica a los datos de entrenamiento y validación.

Con la ayuda del aumento de datos, el modelo se alimenta con imágenes aleatorias con diferencias aleatorias. De esa forma, el modelo se entrena con más datos.

```
1 from tensorflow.keras.preprocessing.image import ImageDataGenerator
2
3 train_gen = ImageDataGenerator(horizontal_flip=True,
4         rotation_range = 45,
5         zoom_range=0.2,
6         height_shift_range = 0.5,
7         width_shift_range = 0.5)
8
9 validation_gen = ImageDataGenerator(horizontal_flip=True,
```

```

10         rotation_range = 45,
11         zoom_range=0.2,
12         height_shift_range = 0.5,
13         width_shift_range = 0.5)
14
15 train_gen.fit(X_train)
16 validation_gen.fit(X_val)

```

4.6. Construyendo el Modelo usando Xception

En lugar de crear un modelo CNN personalizado, se utiliza el modelo Xception previamente entrenado como nuestro modelo base.

Xception es una red neuronal convolucional con 71 capas de profundidad. Es una evolución de la arquitectura Inception desarrollada por Google, que introdujo la idea de usar bloques de convolución que combinan múltiples tamaños de filtro en paralelo.

Preentrenada con más de un millón de imágenes desde la base de datos de ImageNet. La red puede clasificar imágenes en 1000 categorías de objetos. La red ha aprendido representaciones ricas en características para una amplia gama de imágenes. El tamaño de la entrada de imagen de la red es de 299 por 299.

Xception, al ser solo capas de convoluciones, separa el proceso en dos pasos: una convolución por canal (depthwise) seguida de una convolución de punto (pointwise).

La convolución depthwise extrae características dentro de cada canal de forma independiente.

La convolución pointwise mezcla estas características para obtener una representación final combinada.

Esta separación de convoluciones permite que el procesamiento sea más ligero y eficiente sin sacrificar rendimiento.

Se importan las clases para construir el modelo, las capas y la arquitectura preentrenada de Xception, así como el optimizador ".Adam"

```

1 from tensorflow.keras.models import Sequential, Model
2 from tensorflow.keras.layers import Dense, Flatten, Conv2D,
   MaxPool2D, GlobalAveragePooling2D, Dropout
3 from tensorflow.keras.applications.xception import Xception
4 from tensorflow.keras.optimizers import Adam

```

se define el tamaño de batch usado por época y las épocas para el entrenamiento.

```

1 batch_size = 100
2 epochs = 15

```

4.6.1. Definimos el modelo preentrenado (base)

Lo primero es cargar el modelo de Xception, pero se le va a quitar la capa de clasificación original, también se le tiene que indicar que deberá usar los pesos preentrenados en el dataset de ImageNet, finalmente se establece el tamaño de las imágenes de entrada (75 x 75) con los 3 canales de color (RGB)

4.6.2. Agregamos la capa final (head)

Se toma la salida del modelo base y se le aplica una capa de GlobalAveragePooling2D para reducir la dimensionalidad de las características extraídas. Se añade una capa densa con 5 unidades y activación softmax para la clasificación final en 5 clases.

```
1 base = Xception(include_top=False, weights='imagenet', input_shape
    =(75,75,3))
2 x = base.output
3 x = GlobalAveragePooling2D()(x)
4 # Defining the head of the model where the prediction is conducted
5 head = Dense(5, activation='softmax')(x)
```

El modelo final se crea combinando el modelo base y la “cabeza” personalizada (head).

```
1 model = Model(inputs=base.input, outputs=head)
```

4.6.3. Compilación del Modelo

Se compila el modelo utilizando el optimizador Adam con una tasa de aprendizaje (lr) de 0.0001. La función de pérdida es *categorical_crossentropy*, la cual es adecuada para tareas de clasificación multiclase. Se monitoriza la precisión (*accuracy*) como métrica de rendimiento.

```
1 # Compiling the model
2 model.compile(optimizer=Adam(learning_rate=0.0001),
3               loss = 'categorical_crossentropy',
4               metrics=['accuracy'])
5
6 # Fitting the model with train and validation augmented datasets.
7 history = model.fit(train_gen.flow(X_train, y_train, batch_size=
    batch_size),
8                       epochs = epochs,
9                       validation_data = validation_gen.flow
    (X_val, y_val, batch_size=batch_size),
10                      steps_per_epoch = X_train.shape[0] //
    batch_size)
```

Una vez terminado el entrenamiento, se puede graficar el rendimiento del modelo, así mismo, se puede graficar la pérdida en relación a los datos de prueba y validación. 5

```

1 history_df = pd.DataFrame(history.history)
2
3 plt.figure(figsize=(12,4))
4 plt.subplot(1,2,1)
5 plt.plot(history_df['loss'], label='training loss')
6 plt.plot(history_df['val_loss'], label='validation loss')
7 plt.title('Model Loss Function')
8 plt.legend()
9 plt.subplot(1,2,2)
10 plt.plot(history_df['accuracy'], label='training accuracy')
11 plt.plot(history_df['val_accuracy'], label='validation accuracy')
12 plt.title('Model Accuracy')
13 plt.legend();

```

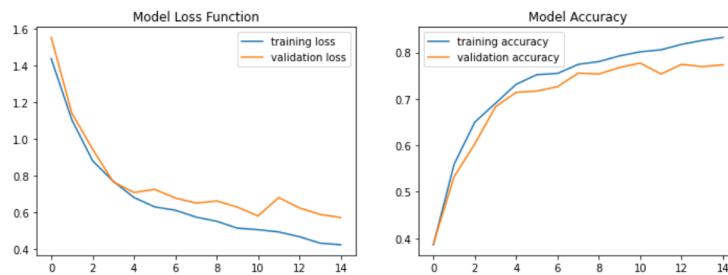


Figura 5: Histograma que muestra la pérdida y el accuracy de los datos de prueba y validación.

Una vez que se tiene cargado el modelo preentrenado ya es posible usar los datos con los que se está trabajando.

Para eso, se usa la instrucción *predict* justo como se muestra a continuación

```

1 from sklearn.metrics import classification_report, confusion_matrix
2 import seaborn as sns
3
4 # Predicting labels from X_test data
5 y_pred = model.predict(X_test)
6
7 # Converting prediction classes from one hot encoding to list
8 # Argmax returns the position of the largest value
9 y_pred_classes = np.argmax(y_pred, axis = 1)
10
11 # Convert test labels from one hot encoding to list
12 y_test_classes = np.argmax(y_test, axis = 1)

```

4.7. Predicción

Ahora se usará el modelo para hacer nuevas predicciones, pasándole algunas imágenes de barcos y este tendrá que determinar a qué categoría corresponden

```

1 path_2 = r'C:\Users\brugi\Documents\Diplomado\ejercicios\Practica_5
2 \train\test_ApKoW4T.csv'
3 test_img = pd.read_csv(path_2)

```

```

3 sample_img = test_img.sample(5)
4
5 # List of image names
6 img_list = list(sample_img['image'])
7
8 sample_img_data = []
9
10 for each in img_list:
11     # Each images' path
12     each_path = os.path.join(path, each)
13     # Read each image
14     each_img = cv2.imread(each_path)
15     # OpenCv default color is BGR. Convert it to RGB
16     each_img = cv2.cvtColor(each_img, cv2.COLOR_BGR2RGB)
17     # Resize the images
18     each_img_resized = cv2.resize(each_img, (128,128))
19     # Save arrays to a list
20     sample_img_data.append(each_img_resized)
21
22 # Converting list to numpy array
23 sample_img_data = np.array(sample_img_data)
24 print('Shape of X: ', sample_img_data.shape)
25
26 # Predicting random 5 images
27 sample_pred = model.predict(sample_img_data)
28 sample_classes = np.argmax(sample_pred, axis = 1)
29
30 # Visualizing the predictions
31 i = 0
32 plt.figure(figsize=(18,9))
33 for each in range(4):
34     i += 1
35     plt.subplot(2,4,i)
36     plt.imshow(sample_img_data[each])
37     plt.xlabel('PREDICTION: ' + str(categories[sample_classes[each]]))
38     plt.xticks([])
39     plt.yticks([])

```

El fragmento de código recién mostrado, arroja los siguientes resultados 6



Figura 6: Predicciones de tipo de barco, usando Xception como modelo

4.8. Análisis de errores.

En este caso se va a analizar principalmente la matriz de confusión que genera Xception con los datos que se le pasaron en esta práctica

4.8.1. Matriz de Confusión

```
1 # Create the confusion matrix
2 confmx = confusion_matrix(y_test_classes, y_pred_classes)
3 f, ax = plt.subplots(figsize = (8,8))
4 sns.heatmap(confmx, annot=True, fmt='.1f', ax = ax)
5 plt.xlabel('Predicted Labels')
6 plt.ylabel('True Labels')
7 plt.title('Confusion Matrix')
8 plt.show();
```

En este caso, la matriz de confusión muestra qué tanto se equivocó o acertó un modelo al intentar reconocer las diferentes categorías de los barcos. Se puede apreciar que la línea diagonal contiene una cantidad considerablemente aceptable de aciertos. 7

Como se puede ver, el caso donde más errores se cometieron es cuando la etiqueta es 4, pero el modelo regresa un 0, es razonable pensar que se debe a que ambas categorías comparten características y que sus diferencias aún no son fácilmente reconocibles por Xception, aún así los resultados con satisfactorios

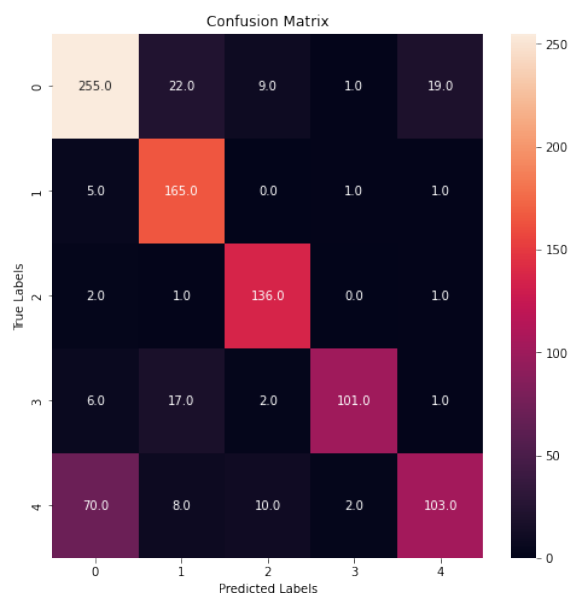


Figura 7: Matriz de confusión creada a raíz de las predicciones de Xception en relación al dataset

4.9. Implementación.

Consulta el Notebook de Jupyter en la carpeta de **Práctica 6** llamada *TF-Barcos.ipynb*